

The lifecycle of an LLM

One model, from random weights to deployed assistant — and the map of everything this book will do to it

Meet a model on the day it is born. Eight billion parameters, initialized from scaled Gaussian noise; its predictions are uniform static, its loss is exactly what Chapter 2 said a know-nothing must score — $\ln(128,256) = 11.76$ nats, a perplexity of 128,256, one guess per vocabulary entry. Ask it anything and it emits confetti. Months later, the same tensor of numbers drafts contracts, debugs code, and declines to help with a few things on purpose. Nothing about its architecture will have changed — same layers, same attention, same shapes. Everything that happens in between is *training data meeting the loop of Chapter 4 in a particular order*, and that order has a name at every stage. This chapter walks the whole biography once, with real numbers at each stage, so that when Parts II–V zoom into single stages you always know where you are standing.

Hold one fact from the start, because it reorganizes how most newcomers think about the field: the stages are wildly unequal in compute and inversely important per FLOP. Pre-training will consume upwards of 95% of the budget and produce something almost no user would recognize as a product. Post-training will consume around one percent and produce nearly everything users actually experience. Efficiency work therefore lives mostly in the big stage; product behavior lives mostly in the small ones; and confusing the two — trying to teach facts in SFT, or fix tone in pre-training — is the classic category error this chapter exists to prevent.

5.1 One model, four ages

ONE 8B MODEL'S LIFE (widths $\propto$ share of training compute)

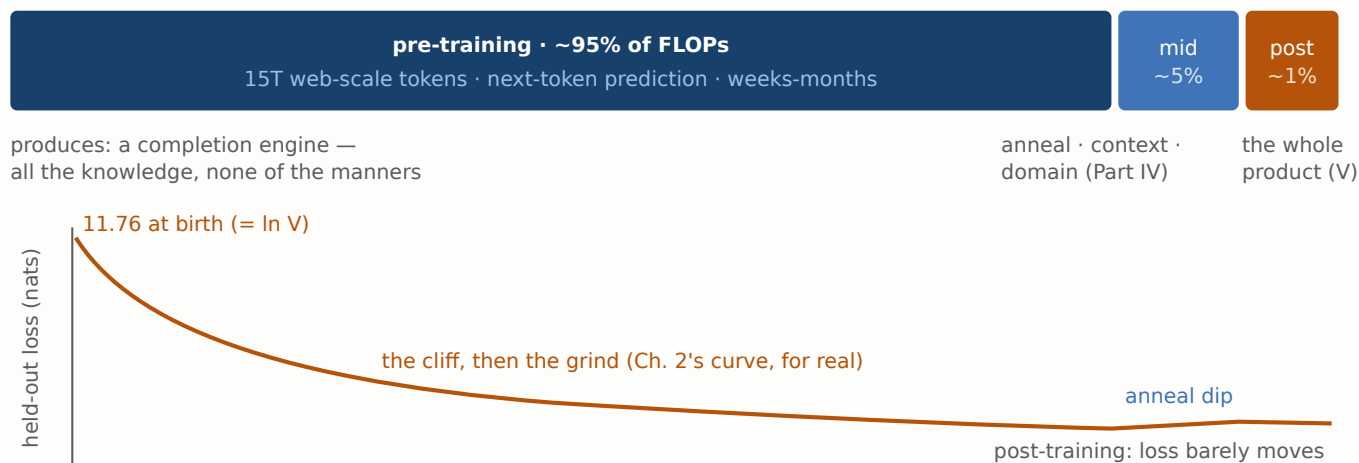


Figure 5.1 — The biography at a glance. Compute lives on the left; product lives on the right. The loss curve does nearly all its falling in pre-training and barely moves in post-training — because post-training's job is not lower loss on the corpus, it is different behavior on prompts. Judging post-training by loss is the first category error this picture prevents.

5.2 Pre-training: the expensive childhood

Pre-training is Chapter 4's loop pointed at as much of the written world as you can clean: our 8B reads ~15 trillion tokens of deduplicated, filtered web text, code, books, and reference material (the corpus engineering is Chapter 10), predicting every next token. The price follows from the one formula this book uses most, Chapter 6's rule of thumb $C \approx 6ND$: with $N = 8B$ and $D = 15T$, about 7.2×10^{23} FLOPs. Make that concrete: at a healthy 40% MFU on H100s, the arithmetic says ~0.5M GPU-hours; Meta's published ledger for its 8B was 1.3M GPU-hours — and the gap between the clean formula and the operational ledger (experiments, restarts, the one-failure-every-three-hours of §4.3) is itself one of this book's recurring lessons.¹ On a 1,000-GPU cluster, that is weeks of continuous running — during which the loss falls from 11.76 down its cliff and into the long grind around ~2 nats, where every further 0.01 costs more than the previous.

What exists at the end is the most misunderstood artifact in the field: a **base model**. It is not an assistant. Prompt it with a question and it may answer, continue the question, or generate four more questions like it — because all three are plausible ways for text to continue. It has absorbed staggering amounts of knowledge and ability (translation, code, reasoning patterns) as *side effects* of compression, but it has no concept that it is in a conversation, no preference for being helpful, and no manners. Everything the public thinks of as "the AI" is still missing — and 95% of the money has been spent. That inversion is why Parts II–III of this book, the efficiency-of-pre-training parts, are the longest: this is the stage where waste is measured in millions.

5.3 Mid-training: the deliberate adolescence

Between the corpus and the product sits a stage that barely had a name five years ago and now has its own Part of this book. Mid-training is still next-token prediction — same loss, same loop — but the *diet* becomes deliberate. Three interventions recur, and our 8B gets all three. The **anneal** (Chapter 20): in the final stretch of the learning-rate glide, feed the highest-quality data you own — curated reference text, textbooks, verified problem sets — where its effect is outsized; the small dip at the end of Figure 5.1's curve is this. **Context extension** (Chapter 21): the staged ladder from 8K to 128K, with the θ -rescaling and dependency-rich data that chapter details — a ~5% token budget, ~0.75T tokens for our model. **Domain and language adaptation** (Chapter 19): continued pre-training on the distribution you actually care about — for a Spanish-and-Portuguese-first model, this is where the corpus tilts — against the catastrophic-forgetting trap that chapter is about. Mid-training's defining property is *leverage*: a twentieth of the budget, aimed late, moves the model more per token than anything since the cliff — and misaimed, it can undo pre-training quality in a week. It is the highest-stakes small stage in the pipeline.

5.4 Post-training: the one percent that ships

Now the transformation users see. Post-training changes the *objective*, not just the data, and it comes in three moves our 8B takes in order. **SFT** (Chapter 23): fine-tune on curated conversations — instruction in, ideal response out, loss only on the response tokens. The scale inversion is the headline: a strong SFT run is millions of examples, on the order of 10^{10} tokens — 0.07% of pre-training compute — and it flips the completion engine into something that answers questions. It teaches format, register, and the assistant persona; per Chapter 27's LIMA lesson, it mostly cannot teach knowledge that pre-training didn't install. **Preference optimization** (Chapter 24): pairs of responses, one preferred;³ DPO-style training pulls toward the preferred and away from the rejected — this is where tone, helpfulness-over-hedging, and the refusal boundary of Chapter 28 get calibrated. **RL on verifiable tasks** (Chapter 25): for reasoning, math, and code, generate solutions, reward the verifiably correct, reinforce — the stage that made the 2025 reasoning wave, and the one with genuine new infrastructure costs (rollouts), which is why Chapter 26's distillation exists as its cheap substitute. Total post-training bill: around one percent of the model's life, for essentially all of its personality.

STAGE	DATA	OBJECTIVE	WHAT THE MODEL GAINS
pre-training	15T tokens, the cleaned world	next token, everywhere	knowledge, language, latent skills
mid-training	~0.75T curated / long / domain	next token, aimed	polish, 128K context, your domain
SFT	~10M conversations (~10 ¹⁰ tok)	next token, answers only	the assistant: format + persona
preferences	ranked response pairs	preferred > rejected	taste, calibration, refusal boundary
RL (verifiable)	problems + checkable answers	reward correct rollouts	reasoning that survives checking

Same weights throughout — only data and objective change. Compute ratio across rows: ~95 : 5 : 1.

Category errors: facts wrong → up (pre/mid data) · tone wrong → down (preferences) · format wrong → SFT.

Figure 5.2 — What each age changes. The debugging heuristic in the last line is worth the whole figure: when a deployed model misbehaves, the row that owns the failure tells you which stage — and which Part of this book — the fix lives in.

5.5 Scaling intuition: the triangle you will negotiate with forever

Three quantities defined our model's life — parameters N , tokens D , compute C — and they are not independent: $C \approx 6ND$ chains them, so fixing any two decides the third. The field's first great empirical result was that quality tracks these quantities smoothly and predictably (scaling laws, Chapter 7); the second was Chinchilla's balance point — for a fixed compute budget, loss is minimized near $D \approx 20N$.² Now look at our biography with those eyes: 8B parameters and 15T tokens is $D \approx 1,875N$ — **ninety-four times** past the "optimal" ratio. Nobody blundered. Chinchilla optimizes *training* cost alone, and a model is trained once but served forever: a smaller model overtrained far past its ratio is slightly worse than the compute-optimal giant, but many times cheaper on every one of the trillions of tokens it will serve. That single economic asymmetry — captured properly in Chapter 7 — explains most sizing decisions of the current era, including why the small models of Chapter 26's distillation family exist at all. The triangle's practical form, which you will use in every planning meeting: *pick the model size your serving economics demand, then buy it all the tokens your budget allows, then spend this book making each token cheaper.*

5.6 Evaluation basics: how you know any of it worked

Every stage above ended with a decision — continue, adjust, or ship — and each decision needs a measurement. Three layers, in increasing cost and decreasing frequency. **Held-out loss:** cross-entropy on data the model never trained on, computable continuously, the heartbeat on every dashboard — supreme for detecting *change*, nearly mute about *capability* (two models 0.02 nats apart can differ enormously in usefulness, and post-training barely moves it at all). **Benchmarks:** standardized task suites — knowledge (MMLU-style), math (GSM8K, AIME-style), code (HumanEval-style), long context (Chapter 21's RULER) — run at checkpoints; comparable across models, endlessly gameable by accident. The accident's name is **contamination:** with 15T tokens scraped from the same internet the benchmarks live on, test items leak into training unless actively hunted (n-gram and embedding overlap, Chapter 9), and a contaminated score is worse than no score because it is confidently wrong. **Humans and arenas:** preference votes on real usage — the closest to truth, the most expensive, the last word before shipping. Chapter 9 turns this into an operating discipline

(which layer runs when, at what cost, gated on what); what belongs in the biography is the habit: *no stage transition without its measurement, and no measurement trusted until contamination has been ruled out.*

5.7 Practical considerations: what actually changes between stages

The infrastructure barely changes; the discipline does. The same cluster, loop, and checkpoints run every stage — what changes is batch composition (documents → packed conversations, Chapter 16), loss masking (everywhere → completions only), evaluation cadence (daily → per-run), and blast radius: a bad pre-training day wastes a day of GPUs; a bad preference run ships a personality defect to every user. Treat stage transitions as releases — checkpoint, evaluate, tag, then proceed — not as Tuesday.

Keep the biography auditable. Six months in, someone will ask why the model refuses fish-recipe questions in Portuguese, and the answer will live three stages back in a filter or a preference batch. The lineage — which corpus version, which SFT set, which preference rounds, which anneal mix (Chapters 10 and 27's provenance discipline) — is what makes that question answerable in an afternoon instead of never.

Common pitfalls

Teaching facts in SFT.

Fine-tuning cannot install knowledge pre-training missed at any reasonable scale; it teaches the model to *confidently phrase* what it doesn't know. Facts are a data problem (Ch. 10, 19), not a post-training problem.

Judging post-training by loss.

Figure 5.1's flat tail is not failure — loss is the wrong instrument for behavior. Win-rates and evals, per Chapter 9.

Skipping the base-model evaluation.

If you only ever evaluate after SFT, you cannot tell whether a weakness is missing capability (expensive fix, early stages) or unexpressed capability (cheap fix, late stages).

Expecting the demo model at 95% spend.

Stakeholder management, but real: budget-holders who see the base model's confetti at the moment most money is gone need Figure 5.1 in advance, not apologies after.

Comparing models across different lifecycle stages.

Your base model vs. their post-trained product is the most common unfair comparison in the field — including in your own ablations.

5.8 The map of this book

You now hold the whole biography, and the rest of the book is this chapter at magnification. **Part II** prices the life you just watched: where the money goes (Ch. 6), the scaling laws behind §5.5's triangle (Ch. 7), the machine-level accounting (Ch. 8), and the measurement discipline of §5.6 (Ch. 9). **Part III** attacks the 95%: the corpus (Ch. 10) and tokenizer (Ch. 11) it reads, the architecture (Ch. 12–13), the parallelism and memory that let it run at all (Ch. 14–15), the packing that stops paying for padding (Ch. 16), and the optimizer dynamics that keep it stable (Ch. 17–18). **Part IV** is the adolescence of §5.3: continued pre-training (Ch. 19), annealing (Ch. 20), context (Ch. 21), and surgery on the model itself (Ch. 22). **Part V** is the one percent that ships: SFT (Ch. 23), preferences (Ch. 24), RL (Ch. 25), distillation as the shortcut through all of it (Ch. 26), the synthetic-data factory (Ch. 27), and the safety boundary (Ch. 28). **Part VI** then runs the entire biography once more with real invoices — the 70B case study (Ch. 29) and the checklist distilled from it (Ch. 30). Every chapter answers the same question this one raised in miniature: *at this stage of the life, where is the waste, and what does removing it cost?*

5.9 Summary

A language model's life is one loop and four ages. Pre-training — ~95% of the compute, $6ND \approx 7 \times 10^{23}$ FLOPs for an 8B on 15T tokens — turns random noise scoring exactly $\ln V$ into a completion engine holding nearly all the knowledge and none of the manners. Mid-training aims a twentieth of the budget deliberately: annealing on the best data during the final glide, the context ladder, domain and language adaptation — maximum leverage per token, and maximum damage when misaimed. Post-training, around one percent of the bill, is nearly all of the product: SFT installs the assistant, preferences install taste and the refusal boundary, RL on verifiable tasks installs reasoning that survives checking — and none of it moves loss much, because its job is behavior, not compression. The sizing triangle $C \approx 6ND$ plus the train-once/serve-forever asymmetry explains the era's overtrained small models — ours sits 94× past Chinchilla's ratio on purpose. Measure every transition with the right instrument — held-out loss for change, decontaminated benchmarks for capability, humans for truth — keep the lineage auditable, and remember the debugging compass of Figure 5.2: facts point early in the life, format points to SFT, taste points to preferences. The rest of this book is this chapter, one stage at a time, with invoices.

References

1. Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. arXiv:2407.21783. 15T-token pre-training; 1.3M reported H100-hours for the 8B; staged post-training with SFT + DPO at millions-of-examples scale.
2. Hoffmann, J., et al. (2022). Training Compute-Optimal Large Language Models (Chinchilla). *NeurIPS*. arXiv:2203.15556. The $D \approx 20N$ compute-optimal ratio behind §5.5 — and the baseline the serving-cost argument deliberately departs from.
3. Ouyang, L., et al. (2022). Training Language Models to Follow Instructions with Human Feedback (InstructGPT). *NeurIPS*. arXiv:2203.02155. The SFT → preferences pipeline of §5.4, as first assembled at scale.

This closes Part I. Next: *Part II — What efficiency means*, beginning with *Chapter 6 — Why training is expensive, and where the money goes*. · [Training Efficient LLMs](#)